ROBO WAR

ESP32 BLE Robot Control Program

1. Introduction

The ESP32 program controls the robot using Bluetooth Low Energy (BLE) communication. Commands sent from a connected web page or Bluetooth client are received by the ESP32 and used to control the left and right motors. The program supports forward, backward, left, right, and stop operations, along with adjustable motor speed.

2. Working Description

- **BLE Communication:** The ESP32 advertises itself as ESP32_Robot and receives movement and speed commands through the Nordic UART Service.
- **Motor Control:** The BTS7960 motor driver signals are controlled through the ESP32 output pins to move the robot in different directions.
- **Obstacle Detection:** The ultrasonic sensor measures the distance in front of the robot. Forward movement is stopped when an obstacle is detected within 15 cm.
- **Reverse Warning:** The buzzer produces an alternating warning sound while the robot moves backward.
- **Safety:** When the Bluetooth client disconnects, the program stops the motors and restarts BLE advertising to allow reconnection.

3. ESP32 Source Code

The following Arduino IDE code is uploaded to the ESP32 microcontroller to implement Bluetooth control, motor movement, ultrasonic obstacle detection, and reverse warning.

```
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>


//==============================
// BLE UUIDs (Nordic UART Service)
//==============================
#define SERVICE_UUID           "6E400001-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"


BLEServer *pServer = NULL;
BLECharacteristic *pTxCharacteristic;
bool deviceConnected = false;


#define BUZZER_PIN 23
unsigned long buzzerTime = 0;
int reverseNote = 0;


//==============================
// Left Motor
//==============================
#define RPWM_L 25
#define LPWM_L 26
```

```
#define REN_L 27
#define LEN_L 14


//==============================
// Ultrasonic Sensor
//==============================
#define TRIG_PIN 4
#define ECHO_PIN 5


//==============================
// Right Motor
//==============================
#define RPWM_R 32
#define LPWM_R 33
#define REN_R 18
#define LEN_R 19


long duration;
float distance;
int speedValue = 150;
String currentCommand = "stop";


// Function Prototypes
void forward();
void backward();
void left();
void right();
void stopMotors();
float getDistance();


//==============================
// BLE Callbacks
//==============================
class MyServerCallbacks: public BLEServerCallbacks {
    void onConnect(BLEServer* pServer) {
      deviceConnected = true;
      Serial.println("Bluetooth Client Connected");
    };

    void onDisconnect(BLEServer* pServer) {
      deviceConnected = false;
      stopMotors();
      Serial.println("Bluetooth Client Disconnected");
      // Restart advertising so we can reconnect
      pServer->getAdvertising()->start();
    }
};

class MyCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
      String rxValue = pCharacteristic->getValue().c_str();


      if (rxValue.length() > 0) {
        rxValue.trim();
        Serial.print("Received Command: ");
        Serial.println(rxValue);


        if (rxValue.startsWith("speed:")) {
          speedValue = rxValue.substring(6).toInt();
          speedValue = constrain(speedValue, 0, 255);
        } else {
          currentCommand = rxValue;
          if (currentCommand == "forward") forward();
          else if (currentCommand == "backward") backward();
          else if (currentCommand == "left") left();
          else if (currentCommand == "right") right();
          else if (currentCommand == "stop") stopMotors();
        }
      }
    }
};
```

```
void setup() {
  Serial.begin(115200);

  // Motor Pins
  pinMode(RPWM_L, OUTPUT); pinMode(LPWM_L, OUTPUT);
  pinMode(REN_L, OUTPUT);  pinMode(LEN_L, OUTPUT);
  pinMode(RPWM_R, OUTPUT); pinMode(LPWM_R, OUTPUT);
  pinMode(REN_R, OUTPUT);  pinMode(LEN_R, OUTPUT);

  digitalWrite(REN_L, HIGH); digitalWrite(LEN_L, HIGH);
  digitalWrite(REN_R, HIGH); digitalWrite(LEN_R, HIGH);

  // Sensor & Buzzer Pins
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  stopMotors();

  // Create BLE Device
  BLEDevice::init("ESP32_Robot");

  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  BLEService *pService = pServer->createService(SERVICE_UUID);

  pTxCharacteristic = pService->createCharacteristic(
                        CHARACTERISTIC_UUID_TX,
                        BLECharacteristic::PROPERTY_NOTIFY
                      );
  pTxCharacteristic->addDescriptor(new BLE2902());

  BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
                                           CHARACTERISTIC_UUID_RX,
                                           BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR
                                         );
  pRxCharacteristic->setCallbacks(new MyCallbacks());

  pService->start();

  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x06);
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();

  Serial.println("BLE Ready! Search for 'ESP32_Robot' on Web Page.");
}

void loop() {
  distance = getDistance();

  // Continuous safety check when moving forward
  if (currentCommand == "forward" && distance <= 15) {
    stopMotors();
    currentCommand = "stop";
    Serial.println("Obstacle detected! Motor stopped.");
  }

  // Active reverse warning sound
  if (currentCommand == "backward") {
    if (millis() - buzzerTime > 250) {
      buzzerTime = millis();
      if (reverseNote == 0) tone(BUZZER_PIN, 1200);
      else if (reverseNote == 1) tone(BUZZER_PIN, 800);
      else noTone(BUZZER_PIN);
```

```
      reverseNote = (reverseNote + 1) % 3;
    }
  } else {
    noTone(BUZZER_PIN);
  }

  delay(20);
}

//==============================
// Motor & Sensor Functions
//==============================
float getDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH, 30000);
  if (duration == 0) return 999;
  return duration * 0.0343 / 2.0;
}

void forward() {
  if (getDistance() <= 15) {
    stopMotors();
    return;
  }
  analogWrite(RPWM_L, speedValue); analogWrite(LPWM_L, 0);
  analogWrite(RPWM_R, speedValue); analogWrite(LPWM_R, 0);
}

void backward() {
  analogWrite(RPWM_L, 0); analogWrite(LPWM_L, speedValue);
  analogWrite(RPWM_R, 0); analogWrite(LPWM_R, speedValue);
}

void left() {
  analogWrite(RPWM_L, 0);          analogWrite(LPWM_L, speedValue);
  analogWrite(RPWM_R, speedValue); analogWrite(LPWM_R, 0);
}

void right() {
  analogWrite(RPWM_L, speedValue); analogWrite(LPWM_L, 0);
  analogWrite(RPWM_R, 0);          analogWrite(LPWM_R, speedValue);
}

void stopMotors() {
  analogWrite(RPWM_L, 0); analogWrite(LPWM_L, 0);
  analogWrite(RPWM_R, 0); analogWrite(LPWM_R, 0);
}
```